# logistics auditor

6 de noviembre de 2026

**Contenido**

**Veridi Logistics: "Last Mile" Delivery Auditor**

1. Set up and Data loading
2. Data cleaning and preprocessing
3. Filtering orders for delivery performance analysis
4. Master dataset
5. Calculating Delivery delays (story 2)
6. Percentage of orders per state (Story 3)
7. The Sentiment Correlation
8. Candidate's Choice — Freight Cost versus Delay Impact

Export master dataset for Visualization

## Veridi Logistics: "Last Mile" Delivery Auditor

**Author:** Lillian Prossy Nabisubi

**Objective:** Audit the Brazilian E-Commerce Public Dataset by Olist to find whether delivery delays are regional or nationwide and analyze their impact on customers.

### 1. Set up and Data loading

In [34]:

```
import pandas as pd
import numpy as np

orders = pd.read_csv('dataset/olist_orders_dataset.csv')
customers = pd.read_csv('dataset/olist_customers_dataset.csv')
reviews = pd.read_csv('dataset/olist_order_reviews_dataset.csv')
products = pd.read_csv('dataset/olist_products_dataset.csv')
order_items = pd.read_csv('dataset/olist_order_items_dataset.csv')
translation = pd.read_csv('dataset/product_category_name_translation.csv')

print(f"Orders: {orders.shape}")
print(f"Customers: {customers.shape}")
print(f"Reviews: {reviews.shape}")
```

Out [34]:

```
Orders: (99441, 8)
Customers: (99441, 5)
Reviews: (99224, 7)
```

### 2. Data cleaning and preprocessing

In [35]:

```
#converting the date columns to datetime objects
date_columns = ['order_purchase_timestamp', 'order_approved_at',
                'order_delivered_carrier_date', 'order_delivered_customer_date',
                'order_estimated_delivery_date']

for column in date_columns:
    orders[column] = pd.to_datetime(orders[column])
```

## 3. Filtering orders for delivery performance analysis

In [36]:

```
valid_orders = orders[~orders['order_status'].isin(['canceled', 'unavailable'])].copy()
print(f"Valid orders: {valid_orders.shape[0]}")

#Average review score per order
average_reviews = reviews.groupby('order_id').agg({
    'review_score': 'mean',
    'review_id':'count'
    }).rename(columns={'review_id':'review_count'}).reset_index()

#Items per order for reducing redundancy
items = order_items.drop_duplicates(subset=['order_id']).copy()
```

Out [36]:

```
Valid orders: 98207
```

## 4. Master dataset

In [37]:

```
master_df = (
    valid_orders
    .merge(customers, on='customer_id', how='inner')
    .merge(average_reviews, on='order_id', how='left')
    .merge(items[['order_id', 'product_id', 'freight_value']],on='order_id', how='left')
    .merge(products, on='product_id', how='left')
    .merge(translation, on='product_category_name', how='left')
)

master_df.head(5)
```

Out [37]:

| | order_id | customer_id | order_status | order_purchase_timest… | order_approved_at | order_delivered_carrier… | order_de |
|---|---|---|---|---|---|---|---|
| 0 | e481f51cbdc54678b7cc4… | 9ef432eb6251297304e76… | delivered | 2017-10-02 10:56:33 | 2017-10-02 11:07:15 | 2017-10-04 19:55:00 | 2017-10-1 |
| 1 | 53cdb2fc8bc7dce0b6741… | b0830fb4747a6c6d20dea… | delivered | 2018-07-24 20:41:37 | 2018-07-26 03:24:27 | 2018-07-26 14:31:00 | 2018-08-0 |
| 2 | 47770eb9100c2d0c44946… | 41ce2a54c0b03bf3443c3… | delivered | 2018-08-08 08:38:49 | 2018-08-08 08:55:23 | 2018-08-08 13:50:00 | 2018-08-1 |
| 3 | 949d5b44dbf5de918fe9c… | f88197465ea7920adcdbe… | delivered | 2017-11-18 19:28:06 | 2017-11-18 19:45:59 | 2017-11-22 13:39:59 | 2017-12-0 |
| 4 | ad21c59c0840e6cb83a9c… | 8ab97904e6daea8866db… | delivered | 2018-02-13 21:18:39 | 2018-02-13 22:20:29 | 2018-02-14 19:46:34 | 2018-02-1 |

5 rows × 25 columns

## 5. Calculating Delivery delays (story 2)

In [38]:

```
# Calculatng the  difference in days (Positive = Late, Negative/Zero = On Time)
master_df['delay_days'] = (
    master_df['order_delivered_customer_date'] - master_df['order_estimated_delivery_date']
).dt.days

# classifying orders into business-friendly statuses
def classify_delivery(row):
    if pd.isna(row['order_delivered_customer_date']):
        return 'Not Delivered'
    elif row['delay_days'] <= 0:
        return 'On Time'
    elif row['delay_days'] <= 5:
        return 'Late'
    else:
        return 'Super Late (>5 Days)'

master_df['delivery_status'] = master_df.apply(classify_delivery, axis=1)
```

```
# Status distribution
print(master_df['delivery_status'].value_counts())
```

Out [38]:

```
delivery_status
On Time                  89936
Super Late (>5 Days)      3764
Late                      2770
Not Delivered             1737
Name: count, dtype: int64
```

## 6. Percentage of orders per state (Story 3)

In [39]:

```
state_summary = master_df.groupby('customer_state').agg(
    total_orders=('order_id', 'count'),
    late_orders=('delivery_status', lambda x: x.isin(['Late', 'Super Late (>5 Days)']).sum()),
    average_delay=('delay_days', 'mean')
).reset_index()

state_summary['late_percentage'] = (state_summary['late_orders'] / state_summary['total_orders']) * 100

# the top 5 worst states for late deliveries
print("Top 10 Worst States for Delivery Delays:")
display(state_summary.sort_values(by='late_percentage', ascending=False).head(10))
```

Out [39]:

Top 10 Worst States for Delivery Delays:

| | customer_state | total_orders | late_orders | average_delay | late_percentage |
|---|---|---|---|---|---|
| **1** | AL | 411 | 85 | -8.707809 | 20.681265 |
| **9** | MA | 736 | 125 | -9.571827 | 16.983696 |
| **24** | SE | 345 | 51 | -10.020896 | 14.782609 |
| **16** | PI | 490 | 66 | -11.306723 | 13.469388 |
| **5** | CE | 1323 | 176 | -10.804535 | 13.303099 |
| **4** | BA | 3344 | 396 | -10.794533 | 11.842105 |
| **18** | RJ | 12698 | 1495 | -11.761215 | 11.773508 |
| **21** | RR | 45 | 5 | -17.292683 | 11.111111 |
| **13** | PA | 969 | 106 | -14.066596 | 10.939112 |
| **7** | ES | 2018 | 214 | -10.496241 | 10.604559 |

## 7. The Sentiment Correlation

In [40]:

```
# Grouping by delivery status to check average review scores
sentiment_check = master_df.groupby('delivery_status').agg(
    avg_review_score=('review_score', 'mean'),
    order_count=('order_id', 'count')
).reset_index()

print(sentiment_check)
```

Out [40]:

```
        delivery_status  avg_review_score  order_count
0                  Late          2.988795         2770
1         Not Delivered          1.817465         1737
2               On Time          4.290589        89936
3  Super Late (>5 Days)          1.738453         3764
```

## 8. Candidate's Choice — Freight Cost versus Delay Impact

*Investigating if heavy or high-freight items suffer worse carrier delays.*

In [41]:

```
# Checking average freight value across delivery statuses
freight_impact = master_df.groupby('delivery_status')['freight_value'].mean().reset_index()
print(freight_impact)
```

Out [41]:

```
        delivery_status  freight_value
0                  Late      21.457733
1          Not Delivered      22.532562
2               On Time      19.957081
3  Super Late (>5 Days)      24.069426
```

## Export master dataset for Visualization

In [42]:

```
master_df.to_csv('master_olist_audit_dataset.csv', index=False)
```